



Data Preprocessing Functions

Essential pandas & scikit-learn functions — with worked examples and output

`read_csv` • `info` • `isnull` • `fillna` • `dropna` • `drop_duplicates` • `get_dummies` • and more



13 Functions, Grouped by Job



Load & Inspect

```
read_csv()  
head() / tail()  
info()  
describe()
```



Find Problems

```
isnull().sum()  
value_counts()
```



Fix & Clean

```
fillna()  
dropna()  
drop_duplicates()  
astype()  
get_dummies()
```



Model Prep

```
train_test_split()  
StandardScaler()
```



FUNCTION 01 · pandas

pd.read_csv()

WHAT IT DOES

Loads data from a CSV file into a DataFrame — usually the very first step.

SYNTAX

```
pd.read_csv("file.csv", sep=",")
```

GOOD TO KNOW

- Use the sep option for other delimiters (tab, ;)
- nrows=100 loads only a sample

pandas

EXAMPLE

```
import pandas as pd

df = pd.read_csv("employees.csv")
df.head()
```

OUTPUT

	Name	Age	City	Salary
0	Sam	25.0	Pune	50000.0
1	Ana	NaN	Delhi	62000.0
2	Ravi	35.0	Pune	NaN
3	Mia	29.0	NaN	58000.0
4	Sam	25.0	Pune	50000.0



FUNCTION 02 · pandas

df.head() / df.tail()

WHAT IT DOES

Previews the first (or last) rows so you can quickly eyeball the data.

SYNTAX

```
df.head(n=5)    df.tail(n=5)
```

GOOD TO KNOW

- Shows 5 rows by default
- A fast sanity check after loading

EXAMPLE

```
df.head(3)
```

OUTPUT

	Name	Age	City	Salary
0	Sam	25.0	Pune	50000.0
1	Ana	NaN	Delhi	62000.0
2	Ravi	35.0	Pune	NaN

pandas



FUNCTION 03 · pandas

df.info()

WHAT IT DOES

Shows column names, data types, and non-null counts in a single view.

SYNTAX

```
df.info()
```

GOOD TO KNOW

- Spot missing data via non-null counts
- Check dtypes before maths or encoding

pandas

EXAMPLE

```
df.info()
```

OUTPUT

```
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype 
---  -
0   Name    5 non-null      str   
1   Age     4 non-null      float64
2   City    4 non-null      str   
3   Salary  4 non-null      float64
dtypes: float64(2), str(2)
```



FUNCTION 04 · pandas

df.describe()

WHAT IT DOES

Summary statistics (count, mean, std, min, quartiles, max) for numeric columns.

SYNTAX

```
df.describe()
```

GOOD TO KNOW

- Add `.round(1)` for cleaner numbers
- min / max can reveal outliers

pandas

EXAMPLE

```
df.describe().round(1)
```

OUTPUT

	Age	Salary
count	4.0	4.0
mean	28.5	55000.0
std	4.7	6000.0
min	25.0	50000.0
25%	25.0	50000.0
50%	27.0	54000.0
75%	30.5	59000.0
max	35.0	62000.0



FUNCTION 05 · pandas

df.isnull().sum()

WHAT IT DOES

Counts the missing (NaN) values in each column.

SYNTAX

```
df.isnull().sum()
```

GOOD TO KNOW

- isnull() returns True/False per cell
- Use .mean()*100 to get percentages

pandas

EXAMPLE

```
df.isnull().sum()
```

OUTPUT

```
Name      0
Age        1
City        1
Salary     1
dtype: int64
```



FUNCTION 06 · pandas

df.fillna()

WHAT IT DOES

Fills missing values — for example with the mean, median, or mode.

SYNTAX

```
df["col"].fillna(value)
```

GOOD TO KNOW

- Numbers -> mean / median
- Categories -> mode()[0]

pandas

EXAMPLE

```
df["Age"] = df["Age"].fillna(df["Age"].mean())  
df["Age"]
```

OUTPUT

```
0    25.0  
1    28.5  
2    35.0  
3    29.0  
4    25.0  
Name: Age, dtype: float64
```




FUNCTION 07 · pandas

df.dropna()

WHAT IT DOES

Removes rows (or columns) that contain any missing value.

SYNTAX

```
df.dropna(axis=0)
```

GOOD TO KNOW

- axis=1 drops columns instead of rows
- Use only when few rows are affected

pandas

EXAMPLE

```
df.dropna()
```

OUTPUT

	Name	Age	City	Salary
0	Sam	25.0	Pune	50000.0
4	Sam	25.0	Pune	50000.0



FUNCTION 08 · pandas

df.drop_duplicates()

WHAT IT DOES

Removes repeated rows, keeping the first occurrence.

SYNTAX

```
df.drop_duplicates()
```

GOOD TO KNOW

- `df.duplicated().sum()` counts them first
- `subset=[...]` checks specific columns

pandas

EXAMPLE



```
df.drop_duplicates()
```

OUTPUT

	Name	Age	City	Salary
0	Sam	25.0	Pune	50000.0
1	Ana	NaN	Delhi	62000.0
2	Ravi	35.0	Pune	NaN
3	Mia	29.0	NaN	58000.0



FUNCTION 09 · pandas

df.astype()

WHAT IT DOES

Converts a column to another data type (int, float, str, category).

SYNTAX

```
df["col"].astype("int")
```

GOOD TO KNOW

- Fill NaN before converting to int
- Use "category" to save memory

pandas

EXAMPLE

```
df["Salary"] = df["Salary"].fillna(0).astype(int)
df.dtypes
```

OUTPUT

```
Name      str
Age       float64
City      str
Salary    int64
dtype: object
```



FUNCTION 10 · pandas

df["col"].value_counts()

WHAT IT DOES

Counts how many times each category appears in a column.

SYNTAX

```
df["col"].value_counts()
```

GOOD TO KNOW

- NaN excluded by default (dropna=False to keep)
- normalize=True gives proportions

pandas

EXAMPLE

```
df["City"].value_counts()
```

OUTPUT

```
City
Pune    3
Delhi   1
Name: count, dtype: int64
```



FUNCTION 11 · pandas

pd.get_dummies()

WHAT IT DOES

One-hot encodes categorical columns into 0/1 indicator columns.

SYNTAX

```
pd.get_dummies(df, columns=[...])
```

GOOD TO KNOW

- dtype=int gives 0/1 (not True/False)
- Best for unordered categories

EXAMPLE



```
pd.get_dummies(df["City"], dtype=int)
```

OUTPUT

	Delhi	Pune
0	0	1
1	1	0
2	0	1
3	0	0
4	0	1

pandas



FUNCTION 12 · scikit-learn

train_test_split()

WHAT IT DOES

Splits features (X) and target (y) into training and test sets.

SYNTAX

```
train_test_split(X, y, test_size=0.2)
```

GOOD TO KNOW

- random_state makes the split repeatable
- stratify=y keeps the class balance

scikit-learn

EXAMPLE

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=1)
print(X_train.shape, X_test.shape)
```

OUTPUT

```
(3, 2) (2, 2)
```



FUNCTION 13 · scikit-learn

StandardScaler().fit_transform()

WHAT IT DOES

Scales numeric features to mean 0 and standard deviation 1.

SYNTAX

```
StandardScaler().fit_transform(X)
```

GOOD TO KNOW

- fit_transform on train; transform on test
- MinMaxScaler scales to 0..1 instead

scikit-learn

EXAMPLE

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled = scaler.fit_transform(X)
print(scaled.round(2))
```

OUTPUT

```
[[-1.19 -1.28]
 [-0.63  0.46]
 [ 0.67 -0.7 ]
 [-0.45 -0.12]
 [ 1.6   1.63]]
```



Quick Reference

Function	Purpose
<code>read_csv()</code>	Load data from a CSV file
<code>head()</code> / <code>tail()</code>	Preview first / last rows
<code>info()</code>	Column types & non-null counts
<code>describe()</code>	Numeric summary statistics
<code>isnull().sum()</code>	Count missing values per column
<code>fillna()</code>	Fill in missing values
<code>dropna()</code>	Drop rows with missing values

Function	Purpose
<code>drop_duplicates()</code>	Remove repeated rows
<code>astype()</code>	Convert a column's data type
<code>value_counts()</code>	Count category frequencies
<code>get_dummies()</code>	One-hot encode categories
<code>train_test_split()</code>	Split into train / test sets
<code>StandardScaler()</code>	Scale features (mean 0, std 1)
<code>fit_transform()</code>	Learn + apply a transform

A clean preprocessing run usually flows top-to-bottom through these functions, then ends with a scikit-learn pipeline.